

Sea Level Requirements

Requirements for the system

ID	Requirement	Description
FR0.1	Load historical data	The clinical trials since (a year) should be loaded into the system in order for it to start working

Identification of Eligible Patients for Clinical Trials

Functional Requirements

ID	Requirement	Description
FR1.1	Extract clinical trial criteria	The LLM must extract inclusion and exclusion criteria from a clinical trial
FR1.2	Convert criteria into logic	The extracted text must be converted into logical expressions (e.g., Age > 18)
FR1.3	Validation of inclusion and exclusion criteria	After the LLM extracts the criteria of inclusion and exclusion a second phase must be undergone, the doctor must validate this results in order for the patients to be elected.
FR1.4	Compare criteria with database	The system must automatically evaluate patient data against the logical criteria of clinical trials
FR1.5	Generate eligibility report	The system must output a list of eligible patients and justification for excluded patients
FR1.6	Saving included patients	After a patient is considered eligible for a trial, he should be paired with it and this information must be persisted

Data Requirements

ID	Requirement	Description
DR1.1	Patient profile fields	The database must contain for each patient the following fields: Age, Diagnosis, Treatment, Stage, etc. As described on the "Automated Structured Data Extraction" section
DR1.2	Criteria mapping	Each criterion must map to corresponding patient attributes in the database

Interface Requirements

ID	Requirement	Description
IR1.1	Clinical trial submission	The clinician must be able to submit several clinical trials into the system.
IR1.2	Clinical trial selection	The clinician must be able to select a clinical trial and view eligibility results.
IR1.3	Reporting interface	The system must display the justification report in a readable format (PDF/HTML).
IR1.4	Clinical Trial Validation interface	A page must be displayed, allowing for the doctor to edit and validate the criteria extracted

Monitoring the Quality of Care

Functional Requirements

ID	Requirement	Description
FR2.1	Retrieve treatment data	The system must retrieve patient treatment records: - Diagnosis date; - First cycle date - Progression date/End of treatment; - Death date; - Status (0/1): - 0: censored (patient didn't had the event until the end of the study); - 1: event occurred (progression or death).
FR2.2	Compute OS and PFS	The system must calculate Overall Survival (OS) and Progression-Free Survival (PFS).
FR2.3	Apply Kaplan-Meier analysis	The system must use Kaplan-Meier survival estimation to analyze outcomes.
FR2.4	Visualize results	The system must generate survival curves and statistical summaries.

For the requirement FR2.1, the extracted fields from the "Automated Structured Data Extraction" phase are mapped with the necessary fields for calculating the Kaplan-Meier:

Extracted field	Necessary field	obs
Date of first treatment	First cycle date	-

Extracted field	Necessary field	obs
Diagnosis date	Diagnosis date	-
Death date	Death date	If it exists for OS event = 1 else event = 0
End date (from treatments)	End of treatment	For PFS event = 0
-	Progression date	For PFS event = 1

Data Requirements

ID	Requirement	Description
DR2.1	Required data fields	Diagnosis date, first cycle date, progression/end date, death date, and status (0/1).
DR2.2	Derived variables	OS = (death date – diagnosis date); PFS = (progression date – first cycle date).

Interface Requirements

ID	Requirement	Description
IR2.1	Analysis dashboard	The clinician must be able to select patient cohorts and view survival graphs.
IR2.2	Report export	Results should be exportable to CSV, PDF, or compatible with SPSS.

Automated Structured Data Extraction

Functional Requirements

ID	Requirement	Description
FR3.1	Select Diaries for extraction	The system must allow one or multiple diaries to be extracted
FR3.2	Extract structured info	The LLM must extract patient data from free-text diaries: - Age / date of birth; - ECOG-PS; describes a patient's level of functioning in terms of their ability to care for themselves, daily activity, and physical ability; - Diagnosis; - Date of Diagnosis; - Molecular Status;

ID	Requirement	Description
		- Stage; - Date of Treatment; - Treatments (First, Second); ? protocolo de quimio? - Start Date; - End Date; - Control In case of fields not existing on the clinical diary the field must be defined to None.
FR3.3	Normalize text	The system must correct typographical and linguistic inconsistencies.
FR3.4	Validation of extracted parameters	After the LLM extracts the parameters the doctor can edit and validate these data.
FR3.5	Store structured profiles	Each structured patient record must be stored in the local database.

Data Requirements

ID	Requirement	Description
DR3.1	Extracted fields	Age / date of birth, ECOG-PS, Diagnosis, Date of Diagnosis, Stage, Molecular Status, Date of Treatment, Treatments, Treatment start date and Treatment end date.
DR3.2	Storage schema	All extracted data must be stored on a postgresql database

Oyster Level Requirements

FR0.1 - Load historical data

All available unstructured clinical diary entries on SClinico for the oncology must be loaded into the PostegreSQL Database.

This will be done through a full load python script where unstructured data is read and stored in the a dedicated table on the DB. Because of the sheer amount of data it is unfeasible for it to be a manual process.

FR1.1 - Extract clinical trial criteria

The LLM must extract inclusion and exclusion criteria from a clinical trial. This process must:

- Parse the unstructured text of the clinical trial

- Identify all eligibility criteria
 - Inclusion Criteria - under the "Inclusion Criteria" section on the clinical trial
 - Exclusion Criteria - under the "Exclusion Criteria" section on the clinical trial
 - If by any chances these sections aren't present the LLM must be capable of classifying the eligibility criteria into the respective categories.

FR1.2 - Convert criteria into logic

After extracting inclusion and exclusion criteria from the clinical trial text (FR1.1), the system must use the LLM to convert each criterion into a structured, machine-readable logical representation. This process must:

- Transform free-text criteria into logical rules (e.g. comparison operators, Boolean conditions)
- This step must output the result in a standardized JSON logic format:

```
{
  "criterion": {
    type : "inclusion/exclusion",
    logic : "[FIELD][OPERATOR][VALUE]"
  }
  ...
}
```

- This conversion must be stored in the PostgreSQL table `clinical_trial_criteria` for further use.

FR1.3 - Validation of inclusion and exclusion criteria

After the extraction and conversion of criteria of inclusion and exclusion (FR1.2) a second phase must be undergone, the doctor must validate this results in order for the patients to be elected. This must be done by:

- Displaying all extracted criteria in a dedicated interface where the doctor can:
 - Review each inclusion and exclusion criterion;
 - Edit incorrect or ambiguous text extracted from the clinical trial;
 - Modify or correct the associated logical rule (operators, field and values).
- The system must only run the patient-trial matching (FR1.3) when all criteria for that trial are validated;
- All clinician modifications must be persisted in PostgreSQL.

FR1.4 - Compare criteria with database

For a given clinical trial, the system must recover and evaluate patient structured data against the logical criteria of clinical trials produced in FR1.2. This process must:

- Retrieve all structured patient records from the PostgreSQL database;
- Retrieve all logical eligibility criteria associated with the selected clinical trial from the `trial_criteria_logic` table.;
- Deterministically evaluate each patient against every inclusion and exclusion rule:
 - Inclusion criteria must all be satisfied;
 - Exclusion criteria must not be satisfied;
 - The evaluation must not rely on randomness or LLM inference at this stage.
- Generate an eligibility decision for each patient:
 - 1 - patient satisfies all inclusion criteria and violates none of the exclusion criteria;
 - 0 - otherwise.

FR1.5 - Generate eligibility report

After the result of FR1.3. the system must output a list of eligible/excluded patients and LLM justification for each decision. The process must be done by:

- Request a justification for the decision from an LLM, having in account:
 - Inclusion and exclusion criteria;
 - Structured fields of the patient;
 - Deterministic decision result.
- Produce a structured dictionary combining the id and name of the patient, previous eligibility decision and the justification produced:

```
eligible_list = {  
    "Patient" : "f{patient_id}-f{Name}",  
    "Decision" : 0/1,  
    "Justification" : "<LLM_prompt>"  
}
```

FR1.6 - Saving included patients

After a patient is considered eligible for a trial, he should be paired with it and this information must be persisted. So it should:

- Store the final eligibility results (decision + justification) in the PostgreSQL table `patient_trial_eligibility` this table connects the patients with the trial.
- Both included and excluded patients must have their decision and justification saved on it. For audit purposes.

FR2.1 - Retrieve treatment data

The system must retrieve all patient treatment-related records required for survival analysis. This process must:

- Load, for each patient, the following structured fields from the PostgreSQL database (previously extracted and validated in the "Automated Structured Data Extraction" phase):
 - Diagnosis date
 - Date of first treatment (mapped to First cycle date)
 - Progression date (if present)
 - End date of treatment (if no progression occurred)
 - Death date (if applicable)
- Automatically derive the Status (0/1) required for Kaplan-Meier calculations based on the available data:
 - OS status rules:
 - 1 - if a death date exists
 - 0 - otherwise (censored)
 - PFS status rules:
 - 1 - if a progression date exists
 - 0 - if progression did not occur and only end-of-treatment is available (censored)
- Apply the following deterministic mapping rules between extracted fields on the "Automated Structured Data Extraction" phase and Kaplan-Meier required fields:

Extracted field	Required KM field	Notes
Date of first treatment	First cycle date	Direct mapping
Diagnosis date	Diagnosis date	Direct mapping
Death date	Death date	OS event = 1 if present, else event = 0
End date (from treatments)	End of treatment	Used when no progression; PFS event = 0
-	Progression date	If present → PFS event = 1

FR2.2 - Compute OS and PFS

After retrieving and mapping all necessary fields in FR2.1, the system must compute the survival metrics Overall Survival (OS) and Progression-Free Survival (PFS) for each patient. This process must:

- Use standardized date formats;
- Apply the following logic for Overall Survival (OS):
 - If death date exists:
 - $OS = (\text{death date} - \text{diagnosis date})$
- Apply the following logic for Progression-Free Survival (PFS):
 - If progression date exists:
 - $PFS = (\text{progression date} - \text{first cycle date})$

FR2.3 - Apply Kaplan-Meier analysis

After computing OS and PFS for each patient (FR2.2), the system must generate survival estimations using the **Kaplan–Meier method**. This process must:

- Compute the Kaplan–Meier survival function deterministically using the standard formula:

$$S(t) = \prod_{t_i < t} \left(1 - \frac{d_i}{n_i}\right)$$

Where:

- $S(t)$, probability of surviving beyond time t .
- t_i , specific time points at which an event occurs.
- d_i , number of events that occur at time t_i .
- n_i , number of individuals who are still at risk of the event just before time t_i
- Ensure the implementation includes:
 - Grouping of patients by survival time to compute d_i and n_i ;
 - Proper handling of censored observations (included in the risk set but not counted as events);
 - Stepwise survival curves (piecewise constant function).
- The Kaplan-Meier estimates must be outputted into a structured format suitable for visualization (FR2.4).
 - Time points;
 - Survival probabilities;
 - Number at risk;
 - Number of events;
 - Censoring information.

FR2.4 - Visualize results

After computing the Kaplan–Meier survival estimates (FR2.3), the system must generate interactive and exportable visualizations for clinician analysis. This process must:

- Produce visualization-ready survival curves with the following characteristics:
 - Stepwise Kaplan-Meier survival function;
 - Distinct curves for OS and PFS;
 - Markers for censored patients (e.g., "+" symbols);
 - Time displayed on the x-axis and survival probability on the y-axis.
- Allow clinicians to:
 - Select the patient cohort or subgroup to be visualized
 - Toggle between OS and PFS views
 - Overlay multiple curves (e.g., by treatment group or stage)
 - Display underlying survival tables (number at risk per time interval)
- Ensure the graphical output can be:
 - Rendered interactively in the system's dashboard
 - Exported in standard formats (PNG/SVG for images, PDF for reports)
 - Accompanied by summary statistics (median survival, events count, censored count)

FR3.1 - Select Diaries for extraction

After the full load of unstructured clinical diaries into the PostgreSQL database (FR0.1), the system must allow the clinician to select on or multiple diaries to undergo structured data extraction. This process must:

- Retrieve and display all available unstructured diary entries stored in the database.
- Provide an interface allowing the clinician to:
 - Select individual diaries manually;
 - Select multiple diaries using batch selection.
- Validate the selection to ensure:
 - Only diaries that have not yet undergone structured extraction (FR3.2-FR3.4) can be selected;
 - Duplicate extractions are prevented;
 - Any missing or corrupted diaries trigger a system warning.
- Forward selected diaries to the structure extraction pipeline (FR3.2-FR3.4) ensuring the system:
 - Records the diary status to "In Extraction" on the database

FR3.2 - Extract structured info

After the selection of relevant unstructured clinical diaries (FR3.1), the system must use an LLM-based extraction pipeline to convert free-text diary content into structured patient data. The process must:

- Process each selected diary independently to ensure reproducibility and traceability.
- Each clinical diary must be injected into the LLM through a deterministic zero-shot prompting pipeline, implemented via a Python script.
- For every diary, the script must construct a standardized prompt template, replacing only the free-text diary content to ensure consistent outputs.
- The prompt must include:
 - Clear extraction instructions
 - A fixed schema the model must respect
 - The raw diary text, inserted verbatim in a designated section, avoiding formatting that may leak metadata or context (e.g. <Diary> <\Diary>)
- The full prompt template should be structured like this:

You are a clinical information extraction assistant.
Your task is to extract structured clinical data from an unstructured free-text clinical diary.

Follow these rules:

- Extract only the information explicitly present in the diary.
- If a field is missing, set its value to null.
- Do not infer or guess clinical data.
- Output only valid JSON, respecting the specified schema.

Schema (all fields must appear in the output, null if missing):

```
{
  "age_or_birthdate": ...,
  "ecog_ps": ...,
  "diagnosis": ...,
  "diagnosis_date": ...,
  "molecular_status": ...,
  "stage": ...,
  "treatments": {
    "1_treatment": {
      "start_date": ...,
      "end_date": ...
    },
    "2_treatment": {
      "start_date": ...,
      "end_date": ...
    }
  }
},
```

```
"control": ...  
}
```

Clinical diary to process:

```
<DIARY>  
{{DIARY_TEXT}}  
</DIARY>
```

Return only the JSON object.

- Extract the following clinical parameters from unstructured text:
 - Demographic information:
 - Age and/or Date of birth.
 - Clinical performance metrics:
 - ECOG-PS (eastern Cooperative Oncology Group - Performance Status).
 - Diagnosis-related information
 - Primary diagnosis;
 - Date of diagnosis;
 - Molecular status;
 - Cancer stage (TNM or equivalent).
 - Treatment information
 - Date of first recorded treatment;
 - Treatments, for each:
 - Start date;
 - End date.
 - Control information.
- Ensure robustness of extraction by:
 - Handling missing information gracefully, if a field is not present in the diary, the system must assign the value null;
 - Logging extraction confidence scores or flags for ambiguous fields
- Produce a structured JSON object for each processed diary in a standardized schema compliant with the database model
- Forward the extracted structured data to the next steps:
 - Normalize text (FR3.3)
 - Validation of extracted parameters (FR3.4)

FR3.3 - Normalize text

The system must correct typographical and linguistic inconsistencies in the extracted clinical fields. This normalization step is executed **after** the first LLM completes the extraction (FR3.2), and it must be performed individually for each extracted diary to ensure data integrity and traceability. The normalization process must:

- Receive as input the structured JSON generated by the previous extraction stage;
- Apply a second deterministic LLM prompt, using a fixed template, to normalize only the textual fields, while preserving numerical and date fields unchanged
- Ensure consistency in spelling, abbreviations, medical terminology, and capitalization
- Prohibit inference, or correction beyond language normalization.
- Ensure that missing fields remain null, without introducing new data.
- Produce a normalized JSON, following the exact same schema as FR3.2, overwriting only the corrected text fields.
- The full normalization prompt template should be structured like this:

```
You are a clinical text normalization assistant.
Your task is to correct typographical and linguistic inconsistencies in the
extracted patient data.
```

Rules:

- Only correct language/format inconsistencies (spelling, abbreviations, capitalization, terminology).
- Do NOT infer missing data or invent information.
- Keep numeric, date, and boolean fields unchanged.
- Output the result strictly in valid JSON, following the schema provided.

Schema (all fields must appear in the output, null if missing):

```
{
  "age_or_birthdate": ...,
  "ecog_ps": ...,
  "diagnosis": ...,
  "diagnosis_date": ...,
  "molecular_status": ...,
  "stage": ...,
  "treatments": {
    "1_treatment": {
      "start_date": ...,
      "end_date": ...
    },
    "2_treatment": {
      "start_date": ...,
      "end_date": ...
    }
  }
}
```

```

    }
  },
  "control": ...
}

```

Here is the extracted JSON from the diary to normalize:

```

<EXTRACTED_JSON>
{{EXTRACTED_JSON}}
</EXTRACTED_JSON>

```

Return only the corrected JSON object.

FR3.4 - Validation of extracted parameters

After the LLM completes the extraction (FR3.2) and normalization (FR3.3) of patient parameters, the system must provide a clinician validation interface to review, edit, and approve the structured data before it is persisted for further analysis. This process must:

- Retrieve the normalized structured JSON for each selected diary from the database.
- Display all extracted fields in a interface, organized by:
 - Demographics (Age, date of birth);
 - Performance status (ECOG-PS);
 - Diagnosis information (diagnosis, date of diagnosis, stage, molecular status);
 - Treatments (start date, end date);
 - Control;
- Allow clinicians to:
 - Edit any incorrectly extracted or normalized field;
 - Add missing data if the diary contains information that was not captured
 - Delete erroneously extracted fields or treatments
 - Approve the validated dataset once all required fields are reviewed
- Ensure that validation status is tracked:
 - Each diary receives a validated = TRUE/FALSE flag
 - Record the timestamp for each validation session
 - Store any clinicians edits as a versioned log to ensure auditability
- Only after validation, the structured data should be used for downstream processes:
 - Patient eligibility evaluation for clinical trials (FR1.3);
 - Survival analysis (FR2.1-FR2.4).
- Provide reproducibility and traceability by:
 - Logging the original extracted JSON, normalized JSON, and the final validated JSON

- Maintaining a history of changes for auditing and quality control.

FR3.5 - Store structured profiles

After extraction (FR3.2), normalization (FR3.3), and clinician validation (FR3.4), the system must persist each structured patient record in the local PostgreSQL database. This process must:

- Store all validated patient records in a dedicated table `patient_profiles` with a schema that includes:
 - Patient identifier;
 - Demographics (Age/Date of birth);
 - Performance status (ECOG-PS);
 - Diagnosis details (diagnosis, date of diagnosis, stage, molecular status);
 - Treatments (start and end dates)
 - Control
- All changes to the unstructured diary, extraction, normalization and validation, must also be persisted in a versioning table `diary_log` as well as the timestamps for these.
- In the `patient_profiles` table it is necessary to ensure that:
 - Only clinician-validated data (FR3.4) is stored
 - Fields not present in the diary are stored as NULL
 - All date fields are stored in standardized ISO format
- The unstructured Diary must have its status updated on the database to "Validated" on the database.